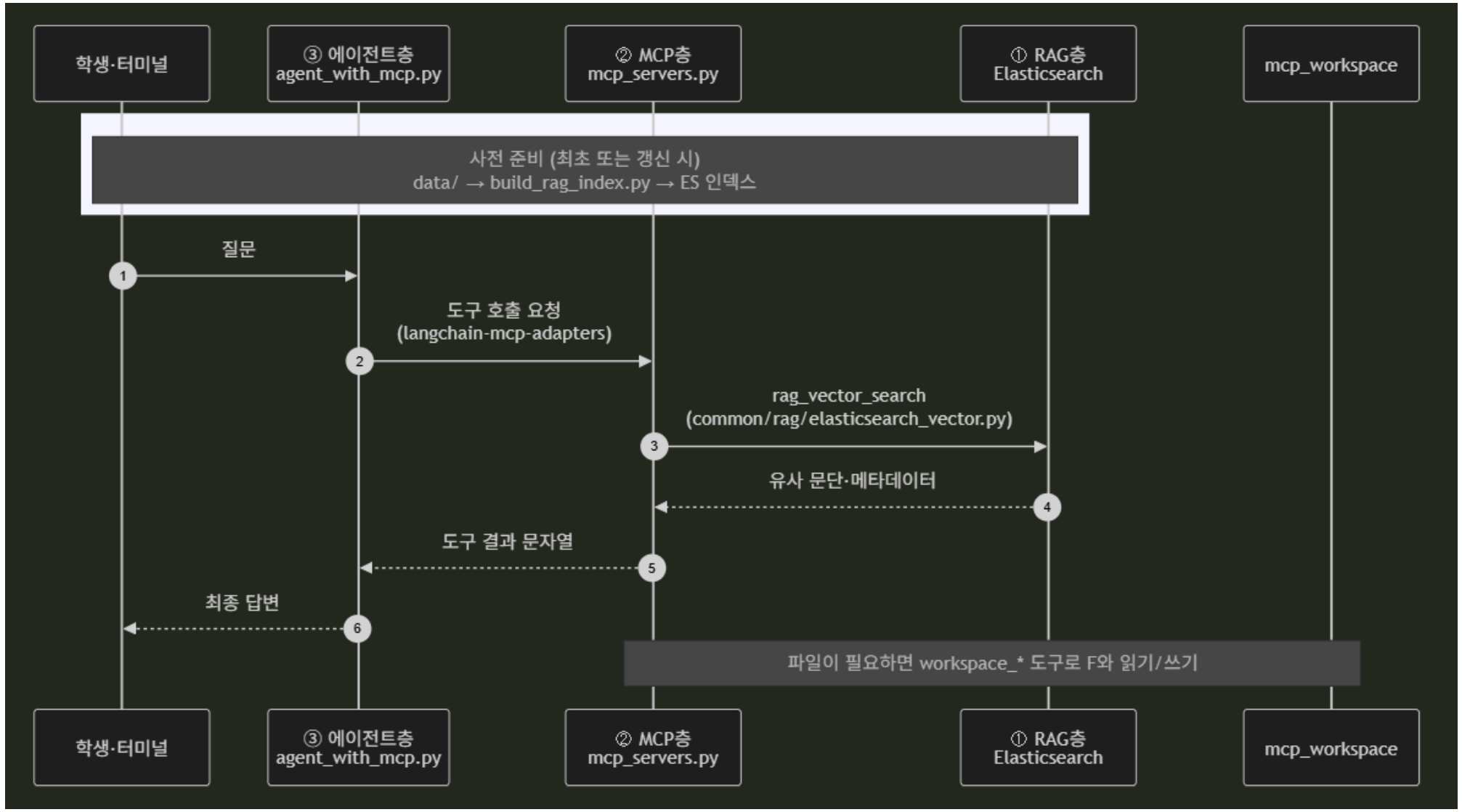
MCP with LangChain — RAG·워크스페이스·에이전트

개념

구분	설명
RAG	`data/` 텍스트를 임베딩해 Elasticsearch에 넣고, 질문과 비슷한 구절을 **벡터 검색**으로 찾습니다.
MCP	LLM이 직접 DB·파일에 접속하지 않고, **표준화된 "도구 서버"**에 요청합니다. 여기서는 검색·파일 I/O가 MCP 도구로 노출됩니다.
Agent	사용자 말을 이해하고 **어떤 도구를 호출할지** 정한 뒤, 결과를 바탕으로 답을 만듭니다. (`langchain.agents.create_agent`)

> 강의 계획에서 말하는 **Agent(도구 호출) Layer**은 `agent_with_mcp.py`에서 `create_agent`로 구현합니다.

아키텍처


학생·터미널
③ 에이전트층
agent_with_mcp.py
② MCP층
mcp_servers.py
① RAG층
Elasticsearch
mcp_workspace
사전 준비 (최초 또는 갱신 시)
data/ → build_rag_index.py → ES 인덱스
1
질문
2
도구 호출 요청
(langchain-mcp-adapters)
3
rag_vector_search
(common/rag/elasticsearch_vector.py)
4
유사 문단·메타데이터
5
도구 결과 문자열
6
최종 답변
파일이 필요하면 workspace_* 도구로 F와 읽기/쓰기
학생·터미널
③ 에이전트층
agent_with_mcp.py
② MCP층
mcp_servers.py
① RAG층
Elasticsearch
mcp_workspace

1. **RAG Layer**
   - `data/` : 용어·설명 텍스트 (`rag-keywords.txt` , `web-keywords.txt` 등)
   - `elasticsearch/` : Docker로 ES·Kibana 실행
   - `common/rag/elasticsearch_vector.py` : ES에 대한 **유사도 검색** 래퍼
2. **MCP Layer**
   - `mcp_servers.py` : FastMCP로 만든 MCP 서버(RAG·워크스페이스 도구). **SSE·Streamable HTTP**로 상시 띄우거나, **stdio**로만 실행할 수 있음
   - RAG 검색 도구 + `mcp_workspace/` 안에서만 동작하는 파일 도구
3. **Agent Layer**
   - `agent_with_mcp.py` : 기본은 **이미 떠 있는 MCP 서버(URL)** 에 붙여 도구를 쓴다. `MCP_USE_STDIO=1` 일 때만 `mcp_servers.py` 를 **자식 프로세스(stdio)** 로 띄운다

사용 라이브러리 (`requirements.txt`)

- **Python**: 3.13 권장
- **langchain**, **langchain-openai**: 채팅 모델·에이전트
- **langchain-mcp-adapters**: MCP 서버와 연결해 LangChain **Tool**로 변환
- **elasticsearch**: 인덱스 구축·검색 클라이언트
- **fastmcp**, **mcp**: MCP 서버 구현·프로토콜
- **python-dotenv**: `.env` 로드

실행 순서 (중요)

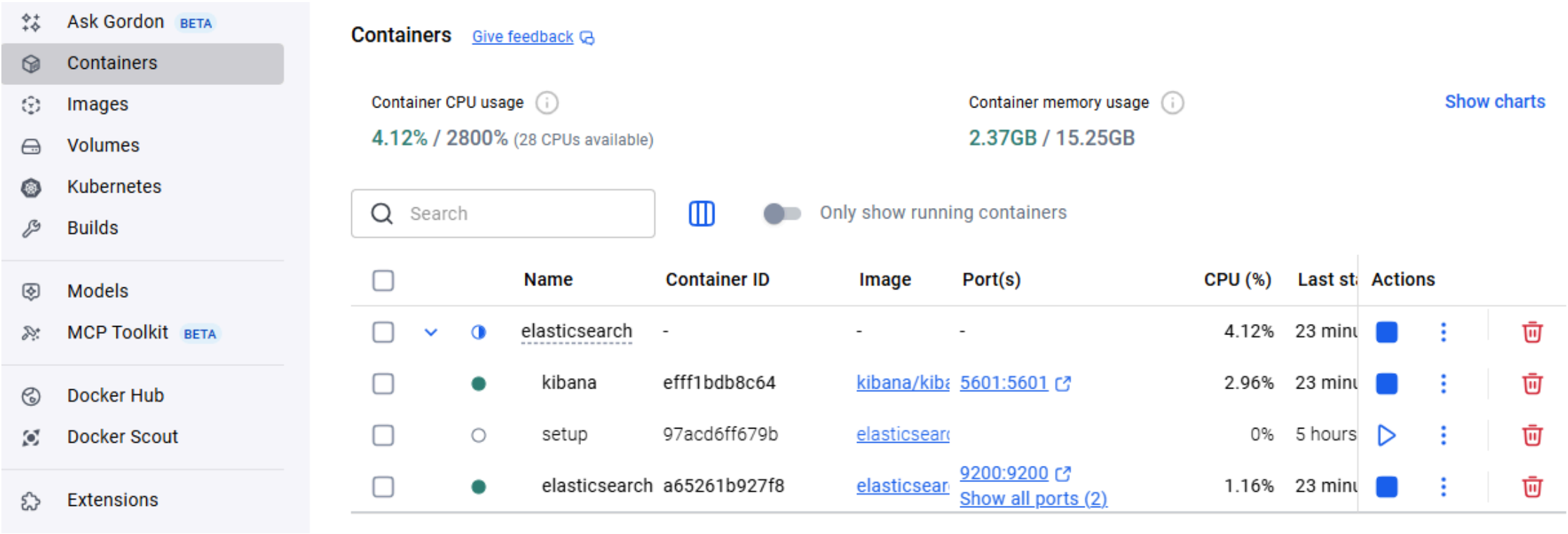
1단계: Elasticsearch 기동

```
cd elasticsearch
docker-compose up -d
```

헬스 확인 예시:

```
curl -u elastic:changeme123! http://localhost:9200/_cluster/health
```

자세한 설명은 `elasticsearch/README.md`를 참고합니다.


Ask Gordon BETA
Containers
Images
Volumes
Kubernetes
Builds
Models
MCP Toolkit BETA
Docker Hub
Docker Scout
Extensions
Containers
Give feedback
Container CPU usage
4.12% / 2800% (28 CPUs available)
Container memory usage
2.37GB / 15.25GB
Show charts
Search
Only show running containers
Name
Container ID
Image
Port(s)
CPU (%)
Last st
Actions
elasticsearch - - - 4.12% 23 minu
kibana efff1bdb8c64 kibana/kiba 5601:5601 2.96% 23 minu
setup 97acd6ff679b elasticsear 0% 5 hours
elasticsearch a65261b927f8 elasticsear 9200:9200 Show all ports (2) 1.16% 23 minu

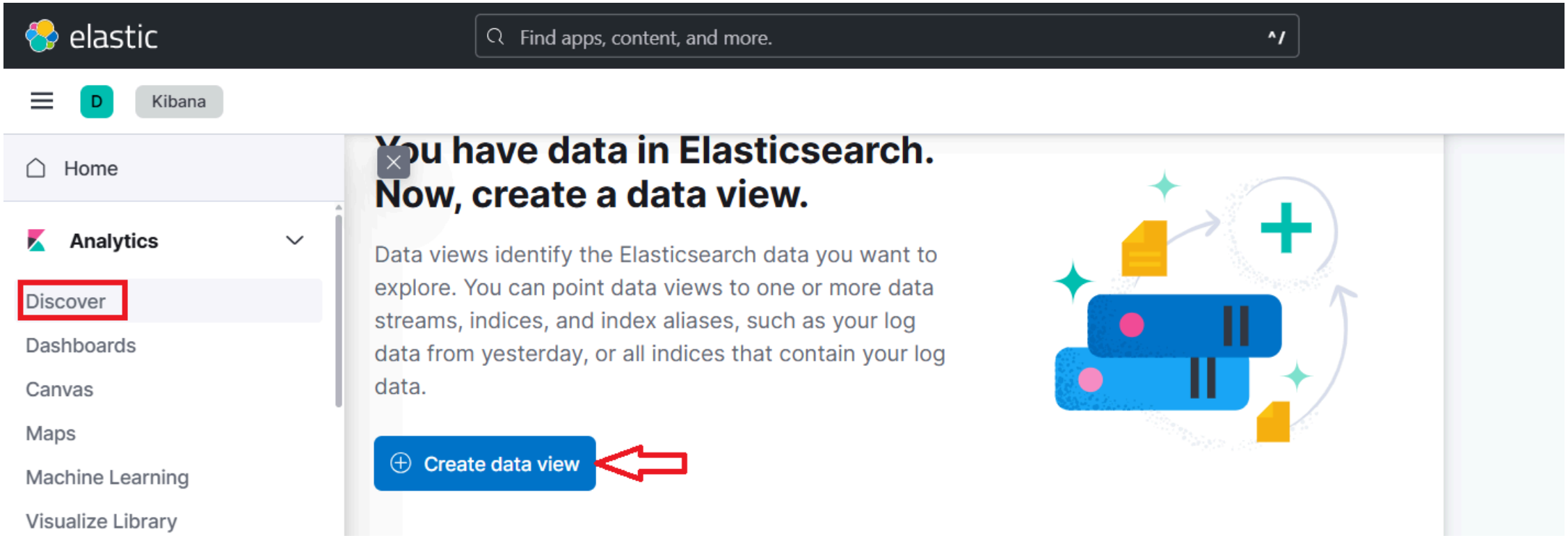
2단계: RAG 인덱스 구축

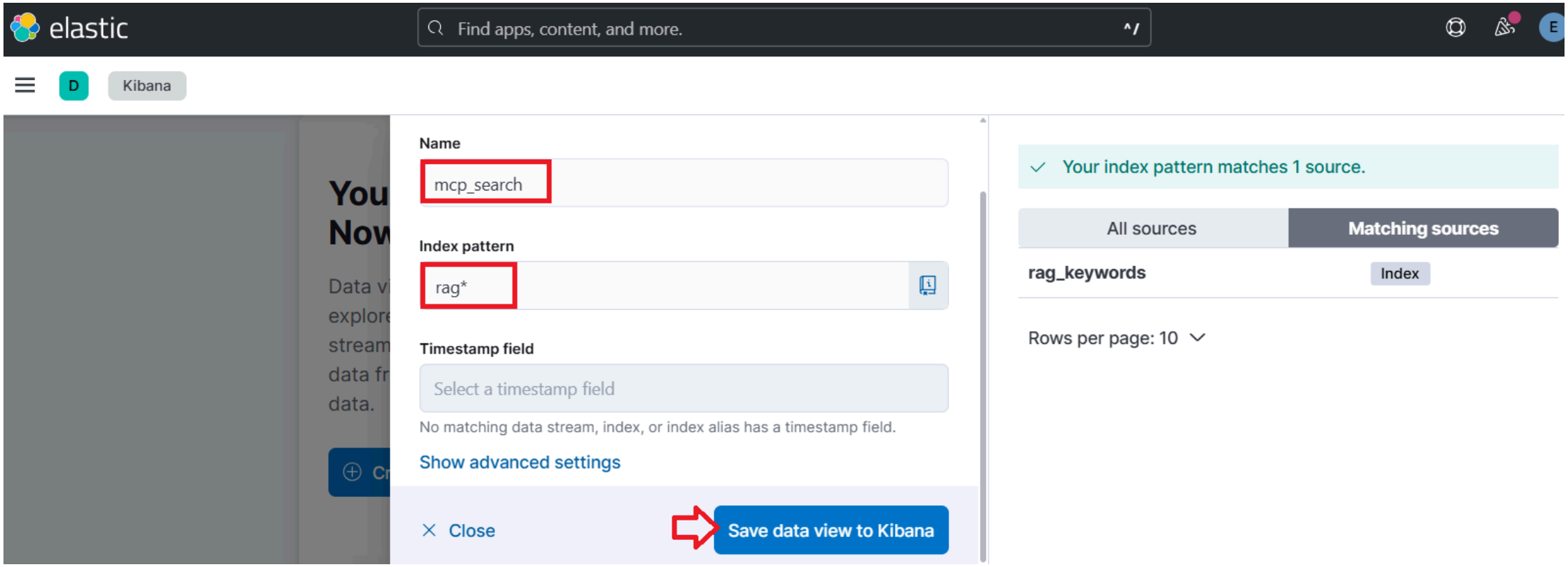
프로젝트 루트로 돌아와:

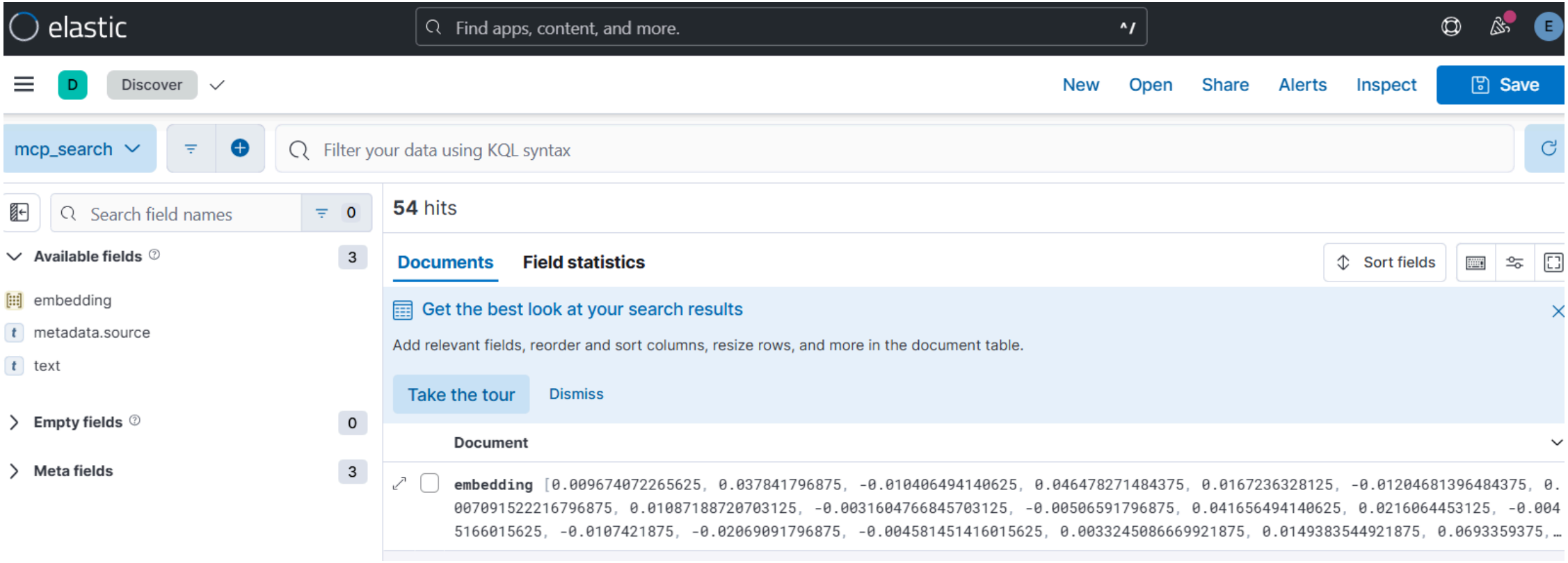
```
python build_rag_index.py --recreate
```

- `data/` 의 지정 파일을 잘라서 임베딩 후 `RAG_INDEX_NAME` 인덱스에 넣습니다.
- 처음이거나 매핑을 바꿨을 때는 `--recreate` 로 인덱스를 다시 만듭니다.

- 로그인 페이지
- `Username` : elastic & `Password` : changeme123!




elastic
Find apps, content, and more.
Kibana
Name
mcp_search
Index pattern
rag*
Timestamp field
Select a timestamp field
No matching data stream, index, or index alias has a timestamp field.
Show advanced settings
Close
Save data view to Kibana
Your index pattern matches 1 source.
All sources
Matching sources
rag_keywords
Index
Rows per page: 10


elastic
Find apps, content, and more.
Discover
New
Open
Share
Alerts
Inspect
Save
mcp_search
Filter your data using KQL syntax
Search field names
0
Available fields
3
embedding
metadata.source
text
Empty fields
0
Meta fields
3
54 hits
Documents
Field statistics
Sort fields
Get the best look at your search results
Add relevant fields, reorder and sort columns, resize rows, and more in the document table.
Take the tour
Dismiss
Document
embedding [0.009674072265625, 0.037841796875, -0.010406494140625, 0.046478271484375, 0.0167236328125, -0.01204681396484375, 0.007091522216796875, 0.01087188720703125, -0.0031604766845703125, -0.00506591796875, 0.041656494140625, 0.0216064453125, -0.0045166015625, -0.0107421875, -0.02069091796875, -0.004581451416015625, 0.0033245086669921875, 0.0149383544921875, 0.0693359375, …

3단계: MCP 서버 상시 기동 (권장)

에이전트(`agent_with_mcp.py`)는 **기본적으로 URL로 MCP 서버에 접속**합니다. 질문할 때마다 서버 프로세스를 새로 띄우지 않으려면, **별도 터미널을 하나 두고 서버를 계속 실행**해 둡니다.

터미널 A — SSE (기본과 맞추기 쉬움)

```
python mcp_servers.py --sse
```

> 로그에 `http://127.0.0.1:8766/sse` 형태로 주소가 나오면 정상입니다. `.env` 의 `MCP_CLIENT_TRANSPORT` 는 기본값 `sse` 를 그대로 두면 됩니다.

```
Problems   Output   Debug Console   Terminal   Ports                                   python

                    Server:      Course – RAG & Workspace, 3.1.1
                    Deploy free: https://fastmcp.cloud

[03/22/26 22:18:51] INFO     Starting MCP server 'Course – RAG & Workspace' with transport 'sse' on http://127.0.0.1:8766/sse   transport.py:273
INFO:     Started server process [10956]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://127.0.0.1:8766 (Press CTRL+C to quit)
```

터미널 A — Streamable HTTP (대안)

```
python mcp_servers.py --streamable-http
```

이 경우 에이전트 쪽 `.env` 에 다음을 맞춥니다.

```
MCP_CLIENT_TRANSPORT=streamable-http
```

포트·호스트는 `MCP_HTTP_HOST`, `MCP_HTTP_PORT`(기본 `127.0.0.1`, `8766`)로 서버·클라이언트 **양쪽**에서 동일하게 둡니다. 경로를 바꿨다면 `MCP_SSE_PATH` / `MCP_STREAMABLE_HTTP_PATH` 또는 전체 URL(`MCP_SSE_URL`, `MCP_STREAMABLE_HTTP_URL`)로 서버와 에이전트가 **같은 엔드포인트**를 보도록 맞춥니다.

추가 옵션:

- `python mcp_servers.py --http` — FastMCP의 `http` 전송(필요 시 사용)
- 경로 덮어쓰기: 서버 실행 전 환경변수 `MCP_SSE_PATH`, `MCP_STREAMABLE_HTTP_PATH`(비우면 FastMCP 기본 경로 사용)

4단계: 에이전트 실행

터미널 B에서 질문을 **명령줄 인자**로 넘깁니다. 에이전트가 필요하면 **MCP 도구**를 호출합니다.

```
python agent_with_mcp.py "임베딩이 뭐야?"
```

> 인자 없이 실행하면 사용법(서버 기동 안내 포함)이 표시됩니다.

```
Problems   Output   Debug Console   Terminal   Ports
(.venv) PS C:\develop\github\course_LLM\6. MCP\3. MCP with API\3. MCP with LangChain> python agent_with_mcp.py "임베딩이 뭐야?"
INFO:root:Question: 임베딩이 뭐야?
INFO:httpx:HTTP Request: GET http://127.0.0.1:8766/sse "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:8766/messages/?session_id=5e61cb3ea0274b0b897510e09aeba82b "HTTP/1.1 202 Accepted"
INFO:httpx:HTTP Request: POST http://127.0.0.1:8766/messages/?session_id=5e61cb3ea0274b0b897510e09aeba82b "HTTP/1.1 202 Accepted"
INFO:httpx:HTTP Request: POST http://127.0.0.1:8766/messages/?session_id=5e61cb3ea0274b0b897510e09aeba82b "HTTP/1.1 202 Accepted"
INFO:root:MCP 도구 목록 로드(sse, 동일 세션): 0.003s, 도구 4개, endpoint=http://127.0.0.1:8766/sse
INFO:httpx:HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
INFO:httpx:HTTP Request: POST http://127.0.0.1:8766/messages/?session_id=5e61cb3ea0274b0b897510e09aeba82b "HTTP/1.1 202 Accepted"
INFO:httpx:HTTP Request: POST https://api.openai.com/v1/chat/completions "HTTP/1.1 200 OK"
INFO:root:에이전트 실행(ainvoke): 6.319s
INFO:root:MCP 세션 전체(연결~종료): 7.221s
INFO:root:Answer: 임베딩(Embedding)은 단어나 문장 같은 텍스트 데이터를 저차원의 연속적인 벡터로 변환하는 과정입니다. 이를 통해 컴퓨터가
텍스트를 이해하고 처리할 수 있게 됩니다. 임베딩은 자연어 처리(NLP)에서 중요한 역할을 하며, 텍스트의 의미를 수치적으로 표현하여 다양한 머
신러닝 작업에 활용됩니다.
(.venv) PS C:\develop\github\course_LLM\6. MCP\3. MCP with API\3. MCP with LangChain>
python
powershell
```

MCP 서버만 단독 실행 (IDE·다른 클라이언트 연동)

에이전트 없이 MCP 프로토콜로만 서버를 띄울 때:

명령	설명
`python mcp_servers.py`	**stdio** MCP (Cursor 등에서 subprocess로 붙일 때)
`python mcp_servers.py --sse`	**SSE** HTTP 서버 (상시 + `agent_with_mcp.py` 기본)
`python mcp_servers.py --streamable-http`	**Streamable HTTP**
`python mcp_servers.py --http`	FastMCP **http** 전송

> 바인딩은 `MCP_HTTP_HOST`, `MCP_HTTP_PORT`로 조정합니다.